

Scalability & Future Plan

Date	29/06/2026
Team ID	Not Assigned
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	Static Dataset — No Real-Time Data Ingestion	The model is trained on a fixed historical dataset (flood dataset.xlsx). It cannot ingest live weather feeds or update predictions dynamically as new rainfall data becomes available.	High
2	Single-Instance Deployment on Free Tier (Render.com)	The Flask app runs on a single Render.com free-tier instance with limited CPU and RAM. Under concurrent user load (20+ users), response times may degrade and the instance may spin down after inactivity.	High
3	No User Authentication or Role-Based Access	The web application has no login system or access control. Any user can access the prediction form, which limits its use in enterprise or government deployments requiring secure, role-based access.	Medium

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Single Gunicorn worker on Render.com free tier. Performance tested up to 20 virtual users via k6 with 95th percentile < 2s.	Migrate to Render.com paid tier or AWS EC2 with auto-scaling. Add multiple Gunicorn workers (--workers 4) and introduce a load balancer (Nginx) for horizontal scaling.
2	Data Storage	Static Excel file (flood dataset.xlsx) loaded at training time. No database backend; model and scaler saved as .save files.	Integrate a PostgreSQL or MongoDB database to store historical predictions, user inputs, and new rainfall records. Enable periodic model retraining with fresh data via scheduled jobs.
3	Performance	XGBoost model loaded into memory at Flask startup via Joblib. Prediction latency is low (~ms) but cold starts on free tier can cause 30–50s delays.	Cache loaded model in memory using Flask application context. Use Redis for session caching. Upgrade to paid hosting to eliminate cold start delays.
4	Security	No authentication, no HTTPS enforcement beyond Render.com default, no input sanitization beyond basic Python try/except.	Add Flask-Login for user authentication. Enforce HTTPS with SSL certificates. Add server-side input validation with WTForms and rate limiting using Flask-Limiter to prevent abuse.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 1	Real-Time Weather API Integration Integrate OpenWeatherMap or IMD API to auto-populate temperature, humidity, and rainfall inputs instead of manual entry.	1–2 Months	Eliminates manual data entry; enables fully automated real-time flood risk alerts.
Phase 2	Mobile-Responsive Progressive Web App (PWA) Rebuild the frontend as a PWA with offline support, push notifications for flood alerts, and GPS-based location detection.	2–3 Months	Extends reach to field workers and disaster responders on mobile devices in flood-prone areas.
Phase 3	Deep Learning Model Upgrade (LSTM / Transformer) Replace XGBoost with an LSTM or Transformer-based time-series model trained on sequential rainfall data for multi-day flood forecasting.	3–5 Months	Improves prediction accuracy for multi-day forecasts; captures temporal rainfall patterns XGBoost cannot model.
Phase 4	GIS-Based Flood Risk Map Dashboard Integrate Folium or Leaflet.js to display a district-level flood risk heatmap across India, updated with live prediction results.	5–6 Months	Enables government analysts and disaster coordinators to visualize flood risk geographically for better resource allocation.